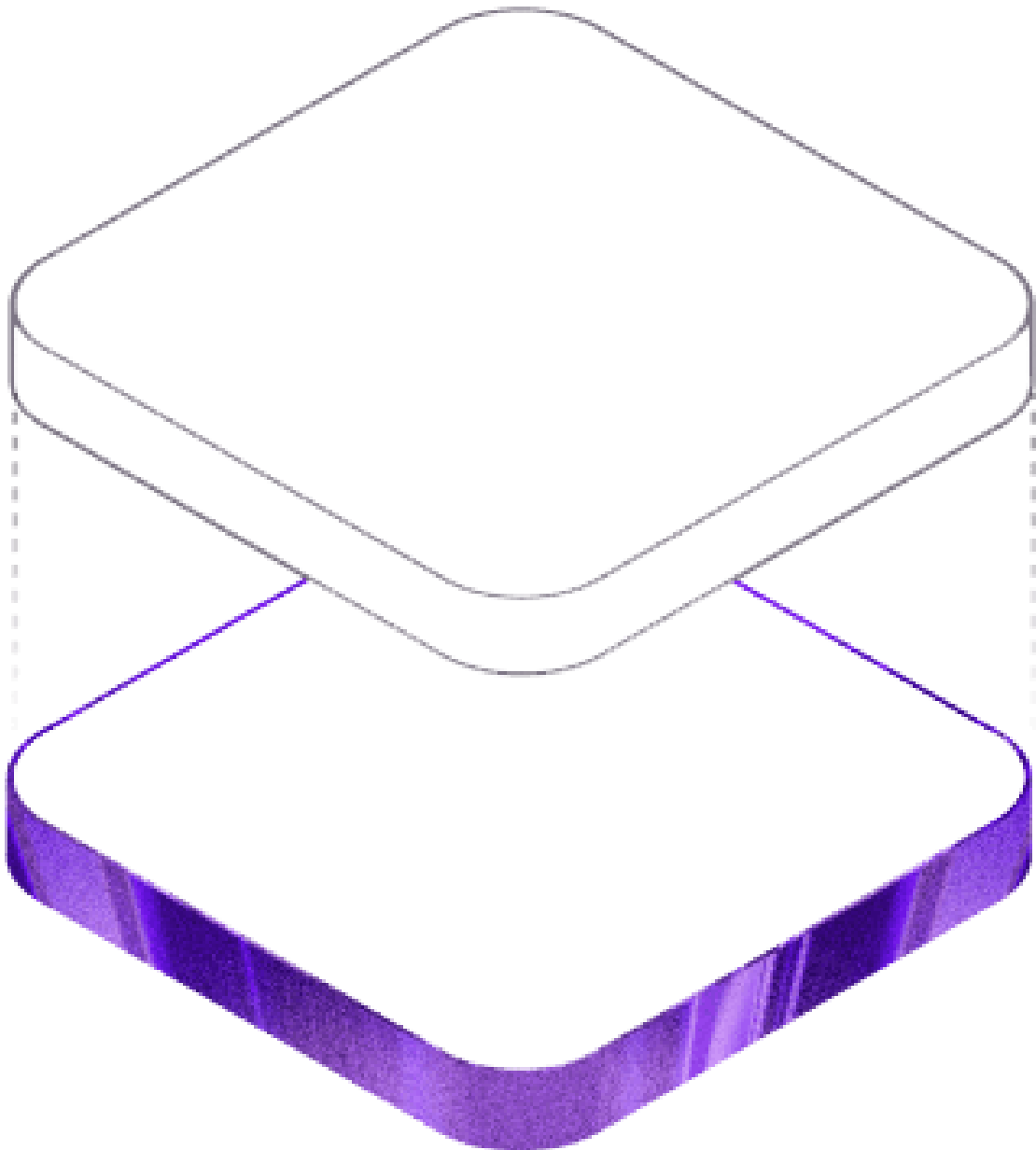


RAPPORT DE STAGE / PROJET DE FIN D'ÉTUDES

Conception et développement de Display Control

Plateforme sécurisée de gestion d'affichage sur Raspberry Pi



Organisme d'accueil

ELASTOMER SOLUTIONS — TANGER

Période : du 3 août au 11 septembre 2026 | Durée : six semaines

Information	Détail
Présenté par	M. NEGAGZA Nizar
Diplôme / filière / établissement	Ingénieur d'État en Génie [à compléter] — ENSI Tanger
Encadrant professionnel	[À compléter]
Encadrant pédagogique	[À compléter]
Année universitaire	2025–2026

Tableau 1 — Informations administratives du stage

Soutenance : [date à compléter] | Année universitaire : 2025–2026

Dédicace

À mes parents et à ma famille, pour leur soutien constant, leur confiance et leurs encouragements tout au long de mon parcours.

À mes enseignants, à mes encadrants et à toutes les personnes qui ont contribué à ma formation et à la réussite de ce projet.

Remerciements

Je tiens à remercier l'ensemble des personnes qui ont contribué au bon déroulement de ce stage et à la réalisation du projet Display Control. Cette expérience m'a permis de travailler sur une problématique concrète mêlant développement web, systèmes embarqués, sécurité applicative, gestion de données et préparation au déploiement.

J'adresse tout particulièrement mes remerciements à l'équipe d'Elastomer Solutions à Tanger pour son accueil, sa disponibilité et la confiance accordée pendant ces six semaines. Les échanges autour des besoins opérationnels, de la gestion des écrans et des contraintes d'exploitation ont donné au projet une orientation pratique et m'ont aidé à relier les choix techniques à leur valeur pour l'entreprise.

Je remercie également mon encadrant en entreprise et mon encadrant pédagogique pour leurs conseils, leurs retours et leur accompagnement. Leurs remarques ont contribué à structurer la démarche, à hiérarchiser les priorités et à conserver une présentation honnête de l'état d'avancement : le cœur logiciel est opérationnel sur le poste de développement, tandis que la validation finale sur Raspberry Pi physique et l'industrialisation restent des étapes distinctes.

Enfin, je remercie les enseignants et toutes les personnes qui m'ont transmis les bases nécessaires en programmation, bases de données, réseaux, sécurité et gestion de projet. Ces connaissances ont été mobilisées tout au long du stage, depuis l'analyse des besoins jusqu'aux tests du dispositif virtuel.

Résumé

Ce rapport présente la conception et la réalisation de Display Control, une plateforme web sécurisée destinée à administrer des écrans pilotés par Raspberry Pi. Le système permet de gérer plusieurs organisations clientes, leurs utilisateurs, leurs appareils, leurs licences, leurs contenus, leurs playlists et leurs affectations. Chaque Raspberry Pi contacte le serveur de manière sortante, s'authentifie avec un certificat propre à l'appareil, remonte son inventaire, reçoit une décision de licence signée et synchronise uniquement le contenu qui lui est autorisé.

La solution repose sur une API ASP.NET Core en C#, une base PostgreSQL, une interface d'administration React/TypeScript et un agent .NET accompagné d'un lecteur React local sur Raspberry Pi. Une attention particulière a été portée à l'isolation entre clients, à l'authentification multifacteur, à l'identité machine par mTLS, à la durée limitée des licences hors ligne, à l'intégrité des fichiers et au comportement sûr en cas d'incertitude. Si l'autorisation n'est pas prouvée, le lecteur n'affiche pas le contenu protégé.

Le parcours principal a été vérifié localement avec Docker et un Raspberry Pi virtuel exécuté sur le poste de développement. L'appareil virtuel a envoyé ses battements de cœur, transmis son nom, son identifiant, ses interfaces IP/MAC, reçu une licence, téléchargé un manifeste et joué le contenu mis en cache. Une coupure du serveur a également confirmé la continuité bornée de la lecture hors ligne et la reprise automatique. Les validations matérielles ARM64, HDMI, systemd et accélération vidéo demeurent à réaliser sur un Raspberry Pi physique.

Mots-clés : Raspberry Pi, affichage dynamique, ASP.NET Core, React, PostgreSQL, mTLS, licence, sécurité, Docker, synchronisation hors ligne.

Abstract

This report presents the design and implementation of Display Control, a secure web platform for centrally managing Raspberry Pi display devices. The system manages multiple isolated customer organizations, users, devices, time-limited licences, media, playlists and assignments. Each Raspberry Pi initiates outbound communication, authenticates with its own client certificate, reports hardware and network inventory, receives a signed authorization decision and synchronizes only approved content.

The solution uses an ASP.NET Core backend written in C#, PostgreSQL, a React/TypeScript administration application, and a .NET device agent with a local React player. Security was treated as a core design constraint through tenant isolation, multi-factor authentication, mutual TLS device identity, signed offline leases, file integrity checks and fail-closed playback behavior.

The main workflow was validated locally using Docker and a Windows-hosted virtual Raspberry Pi. The virtual device successfully sent mutual-TLS heartbeats, reported its hostname and network interfaces, received a licence, synchronized a manifest and played cached content. A temporary server outage confirmed bounded offline playback and automatic recovery. Physical ARM64, HDMI, systemd and hardware-accelerated media tests remain required.

Keywords: Raspberry Pi, digital signage, ASP.NET Core, React, PostgreSQL, mTLS, licensing, security, Docker, offline synchronization.

Table des matières

Remerciements

Résumé

Abstract

Liste des figures

Liste des tableaux

Liste des abréviations

Acronyme	Signification
API	Application Programming Interface
CA	Certificate Authority — autorité de certification
CSR	Certificate Signing Request
CSRF	Cross-Site Request Forgery
ECDSA	Elliptic Curve Digital Signature Algorithm
EF Core	Entity Framework Core
HTTP(S)	HyperText Transfer Protocol (Secure)
MFA	Multi-Factor Authentication
mTLS	Mutual Transport Layer Security
RLS	Row-Level Security
SPA	Single Page Application
TOTP	Time-based One-Time Password
UTC	Coordinated Universal Time
WCAG	Web Content Accessibility Guidelines

Tableau 2 — Acronymes utilisés dans le rapport

Liste des figures

Figure 1 — Architecture fonctionnelle globale de Display Control

Figure 2 — Parcours d' enrôlement d' un appareil

Figure 3 — Décision de licence et synchronisation du contenu

Figure 4 — Frontière entre le serveur, l' agent et le lecteur local

Figure 5 — Scénario exécuté avec le Raspberry Pi virtuel

Liste des tableaux

Tableau 1 — Informations administratives du stage
Tableau 2 — Acronymes utilisés dans le rapport
Tableau 3 — Planification prévisionnelle du stage sur six semaines
Tableau 4 — Comparaison des approches de gestion d'un écran distant
Tableau 5 — Acteurs et responsabilités de Display Control
Tableau 6 — Synthèse des besoins fonctionnels
Tableau 7 — Besoins non fonctionnels principaux
Tableau 8 — Principales entités métier
Tableau 9 — Technologies et rôles dans la solution
Tableau 10 — Organisation principale du dépôt
Tableau 11 — Inventaire remonté par le Raspberry Pi virtuel
Tableau 12 — Niveaux de test et portée des preuves
Tableau 13 — Résultats de la campagne locale du 30 août 2026
Tableau 14 — Synthèse de la livraison du projet
Liste des abréviations
Introduction générale
Chapitre 1 : Contexte général du stage et du projet
1.1 Introduction
1.2 Présentation de l'organisme d'accueil
1.3 Problématique
1.4 Objectifs du projet
1.5 Méthodologie de travail
1.6 Résultats attendus et critères de réussite
1.7 Conclusion
Chapitre 2 : Analyse des besoins et conception
2.1 Introduction

2.2 Étude de l'existant et solution proposée

2.3 Acteurs du système

2.4 Besoins fonctionnels

2.5 Besoins non fonctionnels

2.6 Architecture générale

2.7 Modèle de données

2.8 Identité du Raspberry Pi

2.9 Licence et fonctionnement hors ligne

2.10 Analyse de sécurité

2.11 Conclusion

Chapitre 3 : Réalisation, tests et résultats

3.1 Introduction

3.2 Technologies utilisées

3.3 Organisation du code

3.4 Backend ASP.NET Core

3.5 Authentification et autorisation humaines

3.6 Administration multi-client

3.7 Agent Raspberry Pi et collecte d'inventaire

3.8 Heartbeats et gestion de la licence

3.9 Contenus, playlists et synchronisation

3.10 Lecteur local et états sûrs

3.11 Déploiement Raspberry Pi

3.12 Synthèse de la réalisation

3.13 Stratégie de tests et résultats

3.13.1 Introduction

3.13.2 Stratégie de vérification

3.13.3 Test du Raspberry Pi virtuel

3.13.4 Résultats observés

3.13.5 Anomalies corrigées et limites ouvertes

3.13.6 Validation matérielle restant à réaliser

3.13.7 Apports du stage

3.14 Conclusion

Conclusion générale et perspectives

Références

Annexes

Introduction générale

Les écrans d'information et de communication sont utilisés dans de nombreux environnements : sites industriels, accueils, ateliers, points de vente, salles d'attente ou espaces internes. Leur utilité dépend toutefois de la capacité à mettre à jour le contenu rapidement, à connaître l'état des équipements et à maîtriser la durée pendant laquelle un appareil est autorisé à fonctionner. Une gestion manuelle, écran par écran, devient difficile dès que les équipements sont répartis sur plusieurs réseaux ou plusieurs sites.

Le projet Display Control répond à cette problématique par une plateforme centralisée. L'administrateur prépare le contenu et définit les règles depuis une application web. Les Raspberry Pi n'ont pas besoin d'être découverts sur le réseau local : ils initient eux-mêmes une connexion HTTPS vers le serveur. Cette approche est adaptée aux réseaux séparés par des routeurs et évite d'ouvrir un accès entrant vers les sites où les écrans sont installés.

La difficulté ne se limite pas à envoyer un fichier à un appareil. Il faut savoir quel utilisateur a le droit d'effectuer une action, garantir qu'un client ne puisse pas consulter les données d'un autre, reconnaître un appareil sans se fier uniquement à son adresse IP ou MAC, appliquer une licence avec une date de fin précise, supporter une coupure Internet limitée et empêcher l'affichage lorsque l'autorisation devient incertaine. Le projet a donc été conçu autour d'un principe de sécurité par défaut : en cas de doute, le lecteur se place dans un état sûr.

Le présent rapport expose le contexte du stage chez Elastomer Solutions à Tanger, l'analyse des besoins, les choix d'architecture, la réalisation technique, les tests effectués et les limites encore ouvertes. Il reprend la structure académique du document de référence fourni, mais son contenu est entièrement consacré à Display Control et fondé sur les éléments réels du dépôt.

Chapitre 1 : Contexte général du stage et du projet

1.1 Introduction

Ce chapitre présente le cadre du stage, la problématique traitée, les objectifs et l'organisation du travail sur six semaines. Il permet de comprendre pourquoi la solution associe une application web centrale à un agent installé sur Raspberry Pi et pourquoi les exigences de sécurité et de fonctionnement hors ligne occupent une place importante.

1.2 Présentation de l'organisme d'accueil

Le stage est réalisé au sein d'Elastomer Solutions Maroc SARL, implantée dans la Zone Franche d'Exportation de Tanger. Le groupe développe et fabrique des composants en caoutchouc, en thermoplastique et des pièces bi-matière caoutchouc-plastique destinés principalement à l'industrie automobile. Le site marocain a démarré l'injection de composants en caoutchouc en 2013 ; une nouvelle unité de production à Tanger est entrée en activité en 2016 afin de servir les marchés d'Europe, d'Afrique et des Amériques [1].

Le contexte d'une entreprise disposant d'espaces, d'équipes ou d'équipements à informer rend pertinent un système d'affichage administré à distance. Un écran connecté peut diffuser des informations opérationnelles, des consignes, des messages internes, des visuels ou des vidéos. Pour rester exploitable, la solution doit permettre de savoir quel appareil est actif, quel contenu lui est affecté et jusqu'à quelle date il est autorisé à le présenter.

Repères	institutionnels	vérifiés
Elastomer Solutions Maroc SARL — industrie automobile — Zone Franche d'Exportation, îlot 64, lot 3, 90100 Tanger. Source : site officiel, consulté le 31 août 2026. Le service d'accueil reste à confirmer avec l'encadrant professionnel.		

1.3 Problématique

Une solution d'affichage distribuée doit concilier simplicité d'administration et autonomie locale. Une gestion exclusivement locale oblige un intervenant à se déplacer pour modifier chaque écran. À l'inverse, un lecteur totalement dépendant d'une connexion permanente cesse de fonctionner au moindre incident réseau. L'objectif est donc d'autoriser un fonctionnement hors ligne limité, sans permettre à un appareil de conserver indéfiniment un contenu après la fin de sa licence.

Les identifiants matériels tels que l'adresse MAC, l'adresse IP ou le numéro de série sont utiles pour l'inventaire, mais ils ne constituent pas une preuve cryptographique suffisante. Ils peuvent changer, être masqués ou être copiés. L'appareil doit donc posséder une identité propre, créée pendant un enrôlement contrôlé, puis utilisée pour authentifier les communications ultérieures.

Enfin, une plateforme destinée à plusieurs clients doit empêcher tout mélange de données. Une erreur de filtre dans une requête ne doit pas permettre à un client de voir les appareils, contenus ou licences d'un autre. Ce besoin a conduit à ajouter une séparation au niveau de la base PostgreSQL, en complément des contrôles réalisés par l'API.

1.4 Objectifs du projet

1.4.1 Objectifs fonctionnels

Administrer plusieurs organisations clientes sans mélange de données.

Gérer les utilisateurs, les rôles, les invitations et l'authentification multifacteur.

Enrôler un Raspberry Pi avec un code temporaire à usage unique et lui délivrer un certificat propre.

Collecter le nom de l'appareil, son identifiant, son numéro de série, ses adresses IP/MAC, son système, ses versions et son état.

Créer, renouveler, suspendre, réactiver, révoquer ou transférer une licence associée à un appareil.

Importer des contenus, publier des playlists et les affecter directement ou selon un planning.

Afficher localement les vidéos, images et textes autorisés, ainsi que des états explicites en cas de problème.

Continuer temporairement avec le contenu déjà validé lors d'une coupure réseau, dans la limite d'une preuve signée.

1.4.2 Objectifs techniques et qualité

Séparer les règles métier, l'accès aux données, les interfaces HTTP et les composants de présentation.

Utiliser une base PostgreSQL avec migrations et isolation RLS forcée.

Sécuriser les sessions humaines, les formulaires et les opérations sensibles.

Utiliser mTLS pour les appareils et une signature asymétrique pour les autorisations hors ligne.

Vérifier l'intégrité des contenus téléchargés avant activation dans le cache.

Préparer une publication autonome Linux ARM64 et des services systemd pour Raspberry Pi OS.

Mettre en place des tests unitaires, d'intégration, frontend et navigateur.

1.5 Méthodologie de travail

Le développement a été organisé de manière itérative. Chaque étape devait produire un résultat observable avant d'ajouter la suivante : cadrage des exigences, architecture, modèle de données, identité humaine, identité machine, licences, contenus, lecteur, puis tests de bout en bout. Cette progression réduit le risque de construire une interface complète sur des fondations de sécurité ou de données insuffisantes.

La feuille de route distingue clairement ce qui est implémenté, ce qui est vérifié localement et ce qui nécessite encore un environnement réel. Cette distinction est importante dans un projet de stage : une compilation réussie ne prouve pas le comportement d'un écran HDMI, et un test sur Windows ne prouve pas les permissions systemd ou l'accélération vidéo d'un Raspberry Pi.

Semaine	Travaux principaux	Livrable attendu
1	Analyse du besoin, acteurs, risques et critères d'acceptation	Cadrage et backlog
2	Architecture, modèle PostgreSQL, isolation multi-client	Socle technique et migrations
3	Authentification, rôles, invitations, MFA et administration	Parcours utilisateur sécurisé
4	Enrôlement appareil, certificats, heartbeats et licences	Communication serveur–Raspberry Pi
5	Contenus, playlists, cache, lecteur et paquet ARM64	Chaîne d'affichage complète
6	Tests virtuels/physiques, corrections, rapport et livraison	Démonstration et dossier final

Tableau 3 — Planification prévisionnelle du stage sur six semaines

1.6 Résultats attendus et critères de réussite

Le scénario de réussite retenu est volontairement concret. Un administrateur doit pouvoir créer un client et un appareil, enrôler ce dernier, lui attribuer une licence et un contenu, puis constater que l'écran affiche ce contenu. La plateforme doit montrer l'inventaire remonté et le dernier contact. Lorsque la licence est suspendue ou expirée, le lecteur doit arrêter le contenu protégé et présenter l'état « Non licencié ».

La coupure réseau constitue un second critère essentiel. Si un contenu complet et une autorisation signée encore valable sont présents, la lecture peut continuer pendant une durée bornée, au maximum vingt-quatre heures et jamais après l'expiration réelle de la licence. Si l'agent ne peut plus prouver l'autorisation ou le temps de confiance, il doit échouer de manière sûre.

1.7 Conclusion

Le contexte du stage conduit à un projet transversal qui ne se réduit ni à un site web ni à un programme embarqué. Il combine administration, base de données, protocoles sécurisés, gestion de licences, synchronisation de fichiers et affichage local. Le chapitre suivant présente l'analyse détaillée et les choix de conception retenus pour répondre à ces enjeux.

Chapitre 2 : Analyse des besoins et conception

2.1 Introduction

L'analyse a consisté à transformer l'idée d'un écran administré à distance en exigences précises. Les principaux axes concernent les acteurs, les fonctions attendues, les données manipulées, les états possibles, les limites du fonctionnement hors ligne et les frontières de confiance.

2.2 Étude de l'existant et solution proposée

Une première approche pourrait consister à copier manuellement des fichiers sur une carte SD ou à prendre le contrôle à distance du bureau du Raspberry Pi. Ces méthodes sont simples pour un prototype unique, mais elles deviennent difficiles à maintenir lorsqu'il faut suivre plusieurs appareils, historiser les changements, planifier les contenus ou couper rapidement une autorisation.

Une autre approche serait d'exposer directement un service sur chaque réseau client. Cette solution impose cependant des règles de routeur, de pare-feu et de sécurité entrante. Display Control inverse le sens de communication : le Raspberry Pi initie uniquement des connexions HTTPS sortantes vers le serveur. Le lecteur web local ne s'expose que sur l'interface de boucle locale, ce qui limite sa surface d'attaque.

Approche	Avantages	Limites
Copie manuelle sur carte SD	Très simple pour un écran isolé	Déplacements, aucune supervision, erreurs de version
Prise en main distante	Accès direct au bureau	Ports entrants, manipulation manuelle, faible traçabilité
Service accessible sur chaque Pi	Contrôle direct	Configuration réseau complexe et surface exposée
Display Control	Administration centrale, connexions sortantes, licence et audit	Nécessite serveur, certificats et préparation du Pi

Tableau 4 — Comparaison des approches de gestion d'un écran distant

2.3 Acteurs du système

Acteur	Responsabilités	Limites principales
Administrateur plateforme	Crée et supervise les organisations clientes	Accès global réservé et fortement protégé
Administrateur client	Gère membres, appareils, certificats et licences	Agit uniquement dans son organisation
Gestionnaire de contenu	Importe, approuve, publie	Ne gère pas les rôles ou

	et affecte les contenus	licences sensibles
Lecteur / observateur	Consulte les états autorisés	Pas de mutation administrative
Raspberry Pi en enrôlement	Présente un code à usage unique et une CSR	Aucune identité durable avant validation
Raspberry Pi enrôlé	Heartbeat, synchronisation et lecture locale	Accès limité à son propre périmètre

Tableau 5 — Acteurs et responsabilités de Display Control

2.4 Besoins fonctionnels

Domaine	Fonctions attendues
Comptes	Invitation, connexion, MFA, récupération, rôles et révocation de session
Clients	Création, modification, suspension, réactivation et archivage
Appareils	Enrôlement, inventaire, état, certificat, rotation et révocation
Licences	Création, période de validité, renouvellement, suspension, révocation et transfert
Contenus	Import, contrôle, quarantaine, approbation, archivage et stockage privé
Diffusion	Playlist versionnée, affectation, planning, manifeste et synchronisation
Exploitation	Santé, derniers heartbeats, erreurs sûres, audit et notifications

Tableau 6 — Synthèse des besoins fonctionnels

2.5 Besoins non fonctionnels

Qualité	Exigence retenue
Sécurité	Refus par défaut, moindre privilège, MFA, mTLS, RLS, signatures et audit
Disponibilité	Dernier contenu complet conservé et reprise avec temporisation
Intégrité	Versions immuables, SHA-256, transactions et activation atomique
Maintenabilité	Monolithe modulaire, dépendances épinglées et responsabilités séparées
Portabilité	Conteneurs pour les dépendances et publication autonome linux-arm64

Accessibilité	Interfaces React testées avec Playwright et axe sur les parcours couverts
Traçabilité	Événements d’audit et corrélation sans journaliser les secrets

Tableau 7 — Besoins non fonctionnels principaux

2.6 Architecture générale

L’architecture choisie est un monolithe modulaire. Le backend est déployé comme une application ASP.NET Core unique, mais le code sépare le domaine, l’application, l’infrastructure et l’API. Ce choix réduit le coût opérationnel d’un premier produit tout en évitant de mélanger les responsabilités. Une architecture en microservices aurait ajouté des problèmes de déploiement, de secrets, de cohérence et d’observabilité sans besoin de charge démontré à ce stade.

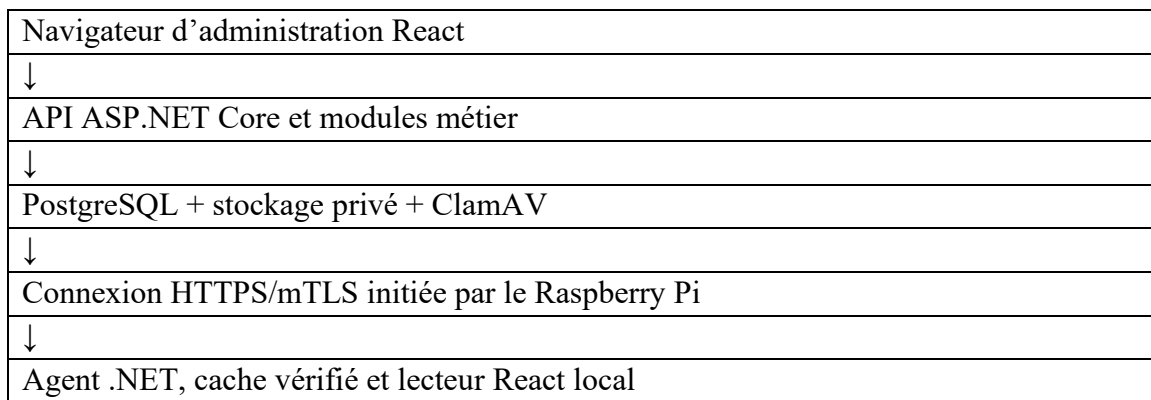


Figure 1 — Architecture fonctionnelle globale de Display Control

L’interface d’administration utilise des sessions humaines et une protection CSRF. L’appareil n’utilise jamais ces cookies : il possède une identité cryptographique distincte. Cette séparation évite qu’un secret destiné à un navigateur devienne l’identité permanente d’un équipement et permet de révoquer un certificat sans affecter les comptes humains.

2.7 Modèle de données

Chaque donnée métier appartenant à un client porte un identifiant de tenant. Les relations utilisent des clés composites ou des contraintes qui empêchent de relier accidentellement des objets appartenant à deux clients différents. PostgreSQL applique en plus la Row-Level Security forcée sur les tables concernées. L’API positionne le contexte du client dans la transaction après avoir vérifié l’appartenance de l’utilisateur ou du certificat.

Entité	Rôle	Règles importantes
Tenant	Organisation cliente	Cycle actif, suspendu ou archivé
Utilisateur / adhésion	Compte et rôle dans un tenant	Invitation, état et MFA
Device	Raspberry Pi administré	Identité, inventaire et état

		de cycle de vie
DeviceCertificate	Credential mTLS	Période, rotation, révocation et liaison appareil
License / LicenseEvent	Autorisation temporelle et historique	Intervalle UTC et événements append-only
Content / ContentVersion	Média importé et version immuable	Quarantaine, approbation et condensat
PlaylistVersion	Ordre de lecture publié	Immuable après publication
DesiredState	Résultat compilé pour un appareil	Version et manifeste déterministes
DeviceHeartbeat	Observation de l'appareil	Séquence, boot ID et réponse rejouable

Tableau 8 — Principales entités métier

2.8 Identité du Raspberry Pi

L' enrôlement commence par un code aléatoire, temporaire et à usage unique créé par l' administrateur. Le Raspberry Pi génère localement une paire de clés ECDSA et envoie uniquement une demande de certificat. La clé privée ne quitte pas l' appareil. Le serveur consomme le code de manière atomique et délivre un certificat client lié au tenant et au device.

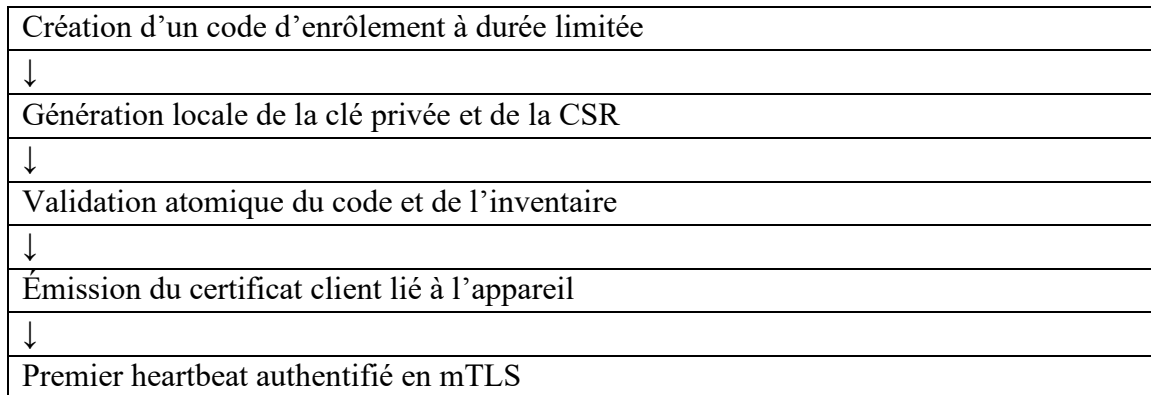


Figure 2 — Parcours d' enrôlement d' un appareil

Après l' enrôlement, le certificat devient la preuve d' identité réseau. Le numéro de série, l' adresse MAC et l' adresse IP restent des informations d' inventaire et de détection d' anomalie. Ils ne remplacent pas la clé cryptographique. Cette distinction explique pourquoi l' application peut afficher ces valeurs sans les considérer comme un mot de passe matériel.

2.9 Licence et fonctionnement hors ligne

La licence stockée en base décrit une période et un état administratif. À chaque heartbeat, le serveur évalue le tenant, l' appareil, son certificat, la licence et le contenu désiré selon l' heure UTC du serveur.

Si l'ensemble est valide, il produit un bail signé en ES256. Ce bail est lié à l'appareil, au certificat, à la licence et au manifeste de contenu.

La date de fin du bail est la plus proche entre la fin réelle de la licence, la fin de l'affectation applicable et l'heure serveur augmentée de l'autorisation hors ligne, plafonnée à vingt-quatre heures. Le lecteur ne se contente donc pas d'une variable locale « `licensed=true` » : il vérifie une preuve signée et bornée dans le temps.

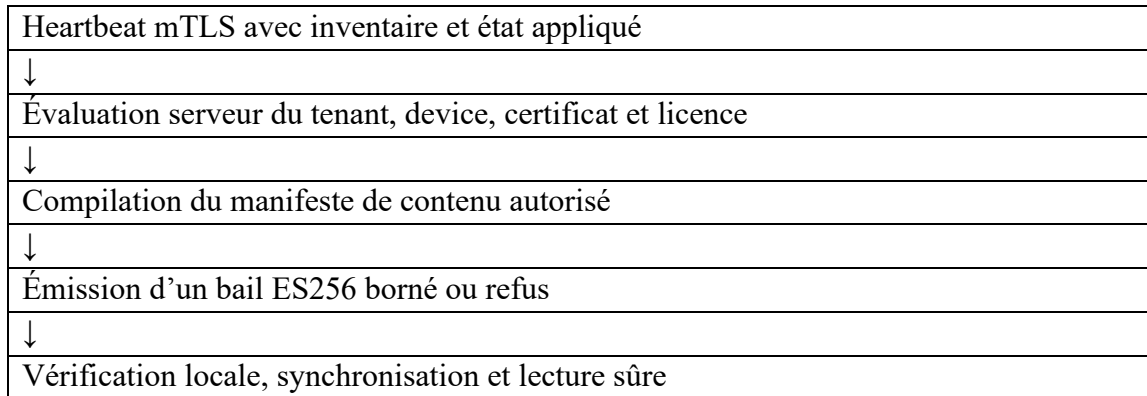


Figure 3 — Décision de licence et synchronisation du contenu

2.10 Analyse de sécurité

Le modèle de menace couvre les comptes humains, les appareils, les fichiers importés, les clés, la base de données et le fonctionnement hors ligne. Les risques les plus importants sont l'accès entre clients, le vol d'un jeton d' enrôlement, l'usurpation d'un appareil, la modification d'une licence, l'introduction d'un fichier malveillant et l'extension artificielle de la durée hors ligne.

Les contrôles incluent l'absence d'inscription publique, les invitations temporaires, les mots de passe hachés, le MFA TOTP, les sessions révocables, les cookies sécurisés, la protection CSRF, la limitation de débit, le mTLS, les signatures asymétriques, l'isolation RLS, le stockage privé et l'analyse antivirus en mode bloquant. Les journaux excluent les mots de passe, codes MFA, jetons bruts et clés privées.

Limite	de	sécurité	assumée
Le système n'est pas un DRM et ne peut pas résister totalement à une personne disposant d'un accès physique et root au Raspberry Pi. La sécurité vise les erreurs, les accès réseau non autorisés et l'exploitation normale ; cette limite doit être expliquée honnêtement.			

2.11 Conclusion

La conception sépare clairement les identités humaines et machines, la décision serveur et l'affichage local, ainsi que les données de chaque client. Ces choix établissent le cadre de la réalisation présentée dans le chapitre suivant.

Chapitre 3 : Réalisation, tests et résultats

3.1 Introduction

La réalisation s'appuie sur un dépôt unique contenant les deux applications React, les projets .NET, les tests, les scripts de déploiement et la documentation. Cette organisation permet de versionner ensemble les contrats utilisés par le serveur, l'agent et les interfaces.

3.2 Technologies utilisées

Composant	Technologie	Utilisation
Backend	.NET 10 / ASP.NET Core	API, sécurité, métier et services HTTP
Accès aux données	Entity Framework Core / Npgsql	Transactions, migrations et PostgreSQL
Base de données	PostgreSQL 18	Données métier, identité, audit et RLS
Administration	React 19 / TypeScript / Vite	Interface web centralisée
Lecteur local	React / TypeScript	Affichage vidéo, image, texte et états sûrs
Agent appareil	.NET Worker / ASP.NET local	Heartbeat, cache, licence et API loopback
Dépendances locales	Docker Compose	PostgreSQL et ClamAV
Tests	xUnit, Testcontainers, Vitest, Playwright, axe	Vérification multi-couche
Cible	Raspberry Pi OS 64-bit / systemd / Chromium	Exécution ARM64 en mode kiosque

Tableau 9 — Technologies et rôles dans la solution

3.3 Organisation du code

Dossier	Responsabilité
apps/admin-web	Application React d'administration
apps/player-web	Lecteur React local et états de sécurité
src/DisplayControl.Domain	Entités, états et règles métier pures
src/DisplayControl.Application	Contrats applicatifs et services transverses
src/DisplayControl.Infrastructure	PostgreSQL, identité, stockage, sécurité et migrations
src/DisplayControl.Api	Contrôleurs HTTP, autorisation et composition de l'application
src/DisplayControl.DeviceAgent	Agent Raspberry Pi, cache et API locale du lecteur

tests	Tests domaine, agent, intégration et navigateur
deploy / scripts	Installation, publication et environnement de test

Tableau 10 — Organisation principale du dépôt

3.4 Backend ASP.NET Core

Le backend expose deux familles d'API. Les routes humaines gèrent les sessions, les clients, les membres, les appareils, les licences, les contenus et les playlists. Les routes device sont séparées et utilisent soit le code d' enrôlement, soit un certificat client. Les erreurs utilisent un format structuré afin que l'interface puisse afficher un message utile sans révéler d'information sensible.

Les contrôleurs délèguent l'accès à PostgreSQL à un contexte EF Core configuré avec des filtres et des transactions tenant. Le contexte de client n'est appliqué qu'après vérification de l'utilisateur ou du certificat. Les opérations sensibles sont enregistrées dans l'audit avec un identifiant de corrélation, le type d'action et un résultat sûr.

Les migrations décrivent le schéma, les contraintes, les index et les politiques RLS. Les tables importantes conservent les événements de licence, les certificats, les heartbeats et les versions publiées. Cette approche facilite l'analyse d'un incident et évite de perdre l'historique lors d'une modification administrative.

3.5 Authentification et autorisation humaines

Le premier administrateur plateforme est créé au moyen d'un jeton d'amorçage à usage unique. Les autres comptes rejoignent une organisation grâce à une invitation. Après la connexion, les administrateurs doivent configurer un second facteur TOTP. Le code de six chiffres est généré par une application d'authentification à partir d'un secret partagé lors de l'activation.

Les sessions sont conservées côté serveur et peuvent être révoquées. Une réinitialisation de mot de passe ou une suspension de client invalide les sessions concernées. Les requêtes qui modifient des données exigent un jeton CSRF. Les actions particulièrement sensibles demandent également une preuve MFA récente, afin qu'une session laissée ouverte ne suffise pas à transférer une licence ou révoquer un certificat.

3.6 Administration multi-client

L'administrateur plateforme crée les organisations clientes et invite leur premier administrateur. Chaque administrateur client voit ensuite uniquement les membres, appareils, licences et contenus de son organisation. L'interface adapte les actions au rôle de la session, mais la sécurité ne dépend jamais du masquage d'un bouton : l'API revérifie systématiquement les permissions.

La suspension d'un client agit à deux niveaux. Les sessions humaines sont révoquées et les appareils cessent d'obtenir une nouvelle autorisation. Cette cohérence évite qu'un client désactivé conserve un accès administratif ou qu'un écran continue à renouveler sa licence côté serveur.

3.7 Agent Raspberry Pi et collecte d'inventaire

L'agent démarre en mode fermé : avant de disposer d'une identité et d'une autorisation valides, il n'expose aucun contenu protégé au lecteur. Lors du premier démarrage, il lit le code d'enrôlement depuis un fichier protégé, génère sa clé privée, demande son certificat puis supprime le code seulement après avoir validé et enregistré la réponse.

L'inventaire envoyé comprend un identifiant de device attribué par le serveur, un nom d'affichage, le numéro de série, le nom d'hôte, le système, l'architecture, les versions logicielles, l'espace disque et les interfaces réseau. Pour chaque interface, le serveur peut enregistrer les adresses IP et la MAC disponibles. Pendant le test virtuel, la plateforme a ainsi reçu le hostname du PC, son IP Wi-Fi 192.168.3.37 et les interfaces virtuelles présentes.

Information	Exemple observé lors du test virtuel
Nom de l'appareil	Virtual Pi
Identifiant serveur	56f7afde-8584-4ddb-8975-74c234a0eb92
Numéro de série de test	VIRTUALPI0001
Hostname	DESKTOP-RE7HDK0
Architecture simulée	x64 sur Windows
IP Wi-Fi observée	192.168.3.37
État lecteur	ready / Playing
Version du manifeste	1

Tableau 11 — Inventaire remonté par le Raspberry Pi virtuel

3.8 Heartbeats et gestion de la licence

Le heartbeat est envoyé périodiquement avec un identifiant de démarrage et un numéro de séquence. Ces valeurs empêchent un ancien message d'écraser un état plus récent. La réponse contient l'heure de confiance, la cadence recommandée, l'état de licence, l'état désiré et, si l'appareil est autorisé, le bail signé nécessaire au fonctionnement local.

La licence suit un cycle de vie complet : active pendant son intervalle, elle peut être renouvelée, suspendue, réactivée, révoquée ou transférée. Chaque changement significatif produit un événement d'historique. Pour un transfert, la destination ne devient pas immédiatement autorisée si une preuve hors ligne valide existe encore sur la source ; le système attend la borne nécessaire pour éviter deux lectures autorisées simultanément.

3.9 Contenus, playlists et synchronisation

Un fichier importé entre d'abord en quarantaine. Le serveur vérifie sa taille, son type déclaré et sa signature réelle, puis s'appuie sur ClamAV en mode bloquant. Si l'analyse n'est pas disponible ou si le fichier est refusé, il ne peut pas être publié. Une version approuvée devient immuable et possède un condensat SHA-256.

La playlist regroupe les versions de contenu dans un ordre déterminé avec une durée et des options de boucle. Une fois publiée, elle devient immuable. L'affectation directe à un appareil ou via un groupe

est ensuite compilée en état désiré. L’agent compare la version reçue à son état local, télécharge les objets absents dans une zone de staging, vérifie chaque condensat puis active atomiquement le nouveau manifeste.

Les téléchargements partiels peuvent reprendre grâce aux requêtes HTTP Range. La longueur et l’entête Content-Range sont vérifiés avant la promotion. En cas de fichier incomplet ou altéré, l’ancienne version complète reste active et l’erreur est signalée au heartbeat suivant.

3.10 Lecteur local et états sûrs

Le lecteur React ne contacte pas directement l’API centrale. Il communique uniquement avec l’agent sur <http://127.0.0.1:8787>. Cette API locale expose l’état courant, le manifeste autorisé et des URLs étroites vers les fichiers actuellement valides. La clé privée, le code d’ enrôlement, le certificat et les jetons backend ne sont jamais transmis au JavaScript du navigateur.

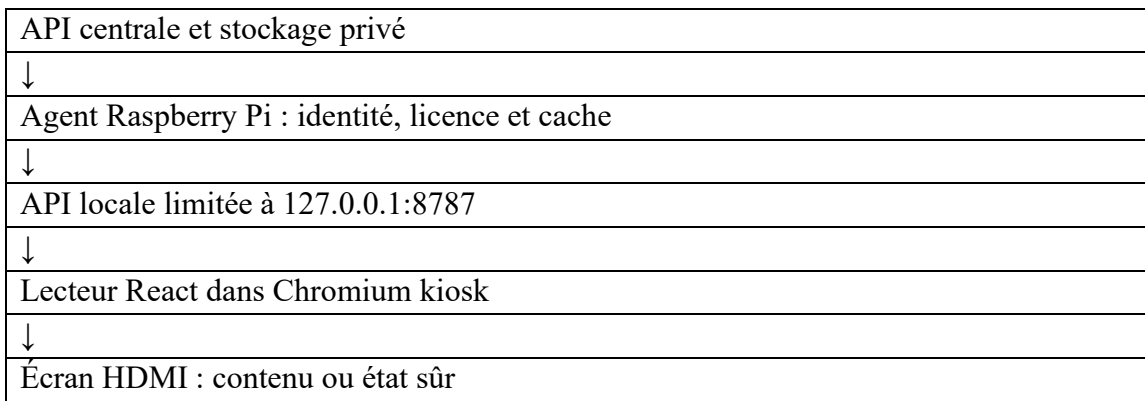


Figure 4 — Frontière entre le serveur, l’agent et le lecteur local

Les états affichés sont distingués pour faciliter l’exploitation : « Non licencié » lorsqu’aucune autorisation n’est valide, « Aucun contenu affecté » lorsque la licence est valide mais qu’aucune playlist ne s’applique, « Synchronisation » pendant la préparation, « Playing » lorsque le manifeste est prêt et un état d’erreur sûr lorsque la lecture échoue.

3.11 Déploiement Raspberry Pi

Le projet prépare une publication .NET autonome pour linux-arm64 contenant l’agent et le lecteur React compilé. L’archive est accompagnée d’un condensat SHA-256. Le script d’installation vérifie l’architecture, crée des comptes de service restreints, installe une version immuable sous /opt, prépare les répertoires d’état et configure les unités systemd.

Le kiosque utilise Chromium avec son bac à sable actif et s’attache à la session graphique Wayland ou X11 disponible. L’agent et le kiosque sont séparés afin que le navigateur ne possède pas les mêmes droits que le composant détenant la clé de l’appareil. La mise à jour utilise un lien current atomique vers une version installée, ce qui prépare un retour arrière contrôlé.

3.12 Synthèse de la réalisation

La réalisation couvre le parcours principal depuis l'administration jusqu'au lecteur local. L'ensemble a été construit avec des frontières de sécurité explicites et des états de panne prévus. Le chapitre suivant présente les tests, les résultats observés et les travaux restant avant une utilisation en production.

3.13 Stratégie de tests et résultats

3.13.1 Introduction

Les tests sont organisés par couche afin que chaque résultat corresponde à ce qu'il peut réellement prouver. Les tests de domaine vérifient les règles sans base ni réseau. Les tests d'intégration utilisent PostgreSQL et le pipeline HTTP. Les tests de l'agent couvrent le stockage d'état, le cache et le temps de confiance. Les tests React et Playwright vérifient les interfaces et certains critères d'accessibilité.

3.13.2 Stratégie de vérification

Niveau	Outils	Éléments vérifiés
Domaine	xUnit	États tenant, device, licence, contenu et planification
Agent	xUnit	État persistant, cache, inventaire et options de sécurité
Intégration	ASP.NET Test Host + Testcontainers	API, PostgreSQL 18, RLS, identité et mTLS
Frontend	Vitest + Testing Library	Composants administration et lecteur
Navigateur	Playwright + axe	Parcours, requêtes, clavier et accessibilité automatisée
Test virtuel	Docker + agent Windows réel	Heartbeat, licence, cache, lecture, coupure et reprise
Matériel	Raspberry Pi 4/5 + écran	ARM64, HDMI, systemd, accélération et endurance — à faire

Tableau 12 — Niveaux de test et portée des preuves

3.13.3 Test du Raspberry Pi virtuel

Un test de bout en bout a été exécuté sur le poste Windows en utilisant Docker Desktop pour PostgreSQL et ClamAV, l'API réelle sur HTTPS et le véritable agent C#. Le poste a représenté le Raspberry Pi avec un numéro de série de développement. Son certificat existant a été utilisé pour établir la connexion mTLS avec l'API.

Démarrage des dépendances Docker et application de la dernière migration PostgreSQL.

Lancement de l'API avec un certificat local valable pour localhost.

Démarrage de l'agent virtuel et envoi d'un heartbeat mTLS accepté en HTTP 200.

Réception d'une licence active et d'une autorisation hors ligne signée.

Chargement du manifeste version 1 et vérification de l'asset texte en HTTP 200.

Arrêt temporaire de l'API, échec contrôlé du heartbeat et maintien de la lecture mise en cache.

Redémarrage de l'API, reprise automatique et renouvellement de l'autorisation.

Docker : PostgreSQL et ClamAV sains
↓
API HTTPS locale prête
↓
Agent virtuel authentifié en mTLS
↓
Licence + manifeste + asset reçus
↓
Coupure : lecture en cache maintenue
↓
Reconnexion : heartbeat et bail renouvelés

•

Figure 5 — Scénario exécuté avec le Raspberry Pi virtuel

3.13.4 Résultats observés

Contrôle	Résultat	Observation
Santé API	Réussi	Endpoints live et ready : healthy
Authentification appareil	Réussi	Heartbeat mTLS en HTTP 200
Inventaire	Réussi	Hostname, serial, IP et MAC enregistrés
Licence	Réussi	Licence active et bail hors ligne borné reçu
Manifeste	Réussi	Version 1, un asset disponible
Lecteur local	Réussi	ready / Playing sur localhost:8787
Coupure serveur	Réussi	Heartbeat en échec, cache toujours accessible
Reconnexion	Réussi	Reprise automatique et nouvelle expiration reçue
Campagne logicielle du 30 août 2026	102/102 .NET ; 8/8 React ; 5/5 navigateur	Compilation, intégration, interfaces et accessibilité automatisée
Raspberry Pi physique	Non exécuté	Matériel ARM64/HDMI encore à valider

3.13.5 Anomalies corrigées et limites ouvertes

Le premier essai virtuel a révélé un défaut de chemin dans le lanceur Windows. Le lanceur et la préparation de l'environnement ont ensuite été corrigés ; le parcours complet est désormais reproductible depuis le paquet compilé prévu pour le Raspberry Pi.

Un scénario d'intégration a mis en évidence une donnée de test incompatible avec la politique de mot de passe renforcée. La fixture a été corrigée sans affaiblir la règle de sécurité, puis toute la campagne a été relancée.

La vérification locale du 30 août 2026 totalise 102 tests .NET, 8 tests React et 5 scénarios Playwright/axe, tous réussis. La compilation Release se termine sans avertissement ni erreur. Ces preuves ne remplacent pas les essais sur Raspberry Pi physique ni la validation d'une infrastructure de recette.

3.13.6 Validation matérielle restant à réaliser

Installer le paquet sur Raspberry Pi 4 ou 5 avec Raspberry Pi OS 64 bits.

Comparer le numéro de série, le hostname et les interfaces IP/MAC avec les commandes du système.

Tester l'affichage vidéo H.264/AAC, image et texte sur un écran HDMI réel.

Vérifier Wayland/X11, Chromium kiosk, systemd et le redémarrage automatique.

Simuler une coupure réseau, une coupure électrique et une horloge incorrecte.

Tester l'expiration et la suspension de licence pendant la lecture.

Tester le manque d'espace disque, le cache corrompu et une synchronisation interrompue.

Réaliser un test d'endurance de 24 à 72 heures avec température et consommation de ressources.

3.13.7 Apports du stage

Ce stage a permis de mobiliser des compétences complémentaires. Sur le plan logiciel, le projet a renforcé la pratique de C#, ASP.NET Core, EF Core, PostgreSQL, React et TypeScript. Sur le plan système, il a introduit les contraintes d'un agent Linux ARM64, des services systemd, d'un navigateur kiosk et d'un cache local résistant aux interruptions.

La dimension sécurité a été particulièrement formatrice. Elle a demandé de distinguer authentification et inventaire, de comprendre les certificats clients, de gérer les durées et signatures de licence, d'isoler les tenants dans la base et de prévoir les états sûrs. La rédaction d'exigences et de critères d'acceptation avant l'implémentation a aussi facilité la vérification et la discussion avec l'encadrement.

Enfin, le test virtuel a montré qu'une démonstration réussie ne suffit pas : l'environnement de lancement, les scripts, les certificats et les données de test font partie du produit. Une anomalie de script ou une fixture obsolète peut masquer un fonctionnement correct ou produire une fausse alerte. La reproductibilité constitue donc un livrable à part entière.

3.14 Conclusion

Le parcours logiciel principal est démontré localement, y compris la perte et la reprise du serveur. Les résultats soutiennent la pertinence de l'architecture et valident le comportement du véritable agent sur le poste. La dernière étape technique du stage reste la validation sur le matériel cible et dans un environnement de recette représentatif.

Conclusion générale et perspectives

Display Control apporte une réponse structurée au besoin de gérer des écrans Raspberry Pi à distance. La plateforme centralise les utilisateurs, appareils, licences, contenus et affectations, tandis que chaque appareil conserve une autonomie locale strictement bornée. L'identité mTLS, la signature des baux hors ligne, l'isolation PostgreSQL et le cache vérifié évitent de réduire la sécurité à un simple identifiant matériel ou à une variable locale.

Le stage a abouti à un socle fonctionnel important : administration multi-client, authentification avec MFA, enrôlement des appareils, inventaire, cycle de licence, contenus versionnés, playlists, synchronisation et lecteur local. Le test virtuel a confirmé le chemin critique et le comportement pendant une coupure. La solution est donc démontrable et prête pour une campagne matérielle, mais elle n'est pas encore déclarée prête pour la production.

À court terme, les priorités sont la correction du lanceur virtuel, l'actualisation de la fixture de mot de passe et l'exécution sur un Raspberry Pi connecté à un écran. À moyen terme, il faudra préparer un environnement de recette avec DNS, TLS public, stockage privé, SMTP, supervision et sauvegardes, puis conduire les tests de charge, d'accessibilité manuelle et de sécurité. Enfin, un mécanisme complet de mise à jour signée et de retour arrière renforcera l'exploitation d'une flotte d'appareils.

Au-delà du résultat technique, ce projet montre l'importance de relier chaque fonctionnalité à une preuve et chaque preuve à un environnement précis. Cette démarche permet de présenter au mentor une réalisation ambitieuse sans confondre prototype local, validation matérielle et exploitation industrielle.

Références

1. Microsoft, documentation ASP.NET Core et Entity Framework Core, versions .NET 10.
2. PostgreSQL Global Development Group, documentation PostgreSQL 18 — Row Security Policies.
3. React Documentation, React 19 ; Vite Documentation, Vite 8.
4. OWASP, Application Security Verification Standard 5.0 et recommandations CSRF/authentification.
5. IETF, TLS 1.3, certificats X.509 et JSON Web Signature ES256.
6. Raspberry Pi Documentation, Raspberry Pi OS, Chromium et configuration de l’affichage.
7. Documentation interne du projet : exigences, architecture, modèle de données, API, modèle de menace, stratégie de test et procédures de déploiement.

Annexe A : Parcours de démonstration

Démarrer Docker Desktop, PostgreSQL et ClamAV.

Lancer l'API HTTPS et vérifier les endpoints de santé.

Se connecter avec un compte de démonstration et compléter le code MFA si nécessaire.

Afficher l'appareil Virtual Pi et contrôler son hostname, son numéro de série et ses interfaces.

Vérifier que la licence est active et que la playlist est affectée.

Ouvrir le lecteur local sur <http://localhost:8787> et constater l'état Playing.

Couper l'API brièvement, vérifier la continuité bornée, puis redémarrer et observer la reprise.

Attention

Les comptes, secrets TOTP, certificats privés et données du dossier .data ne doivent jamais être placés dans le rapport ni transmis avec le code source. Utiliser uniquement des données de démonstration temporaires.

-

Annexe B : État de livraison

Élément	État au 31 août 2026
Code source backend et frontend	Présent
Agent et lecteur Raspberry Pi	Présents
Migrations PostgreSQL et RLS	Présentes
Scripts de test terrain et publication ARM64	Présents
Test virtuel complet	Réussi avec limites documentées
Validation Raspberry Pi physique	À réaliser
Infrastructure de production	À préparer
Rapport Word et PDF	Générés le 31 août 2026

Tableau 14 — Synthèse de la livraison du projet